

Reducing Runtime Overhead Caused by JIT (Just-In-Time) Compilation

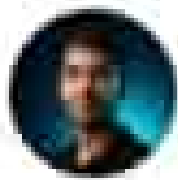
JIT compilation compiles Intermediate Language (CIL/IL) into machine code at execution time. To eliminate or minimize startup latency and CPU overhead, the following techniques and features are used:

Ahead-Of-Time (AOT) Compilation / Native AOT: Compiles CIL directly into native machine code prior to execution. This eliminates the JIT compiler entirely at runtime, drastically reducing startup time and memory footprint.

ReadyToRun (R2R) Images: A hybrid approach where binaries are pre-compiled into native code, but still contain CIL as a fallback for scenarios where native code cannot be executed.

Tiered Compilation: The runtime uses fast, unoptimized JIT pass (Tier 0) to start execution immediately, and then re-compiles frequently executed methods ("hot paths") using an optimizing JIT compiler (Tier 1).

Profile-Guided Optimization (PGO): Collects runtime data about execution patterns to allow the JIT compiler to generate highly optimized native code tailored to actual application usage.



Abdallah El-Mala • You

Faculty of computer science | 6 October University | Interesse...

12m •

١. إيه اللي كان بيحصل زمان؟ (قبل الـ .NET)

زمان لما كنت بتكتب كود بلغة زي C أو ++C، الكمبيوتر كان بياخد الكود ده وبترجمه مرة واحدة بس للغة الآلة (Machine Code) - اللي هي الـ 0 والـ 1 بتاعة البروسيسور.

الخطو في كده: البرنامج يبقى سريع جداً ومبيستهلكش وقت عشان يشتغل، لأن البروسيسور فاهمه جاهز المشكلة:

لو عملت برنامج لـ Windows مع بروسيسور Intel، مش هيشغل على Mac
لأ لما تعيد ترجمته وتطبقه من جديد.

انت المسئول عن الميموري| يعني لو نسيت تقضي مساحة في الـ RAM، البرنامج ممكن يهتج الجهاز أو يحصل فيه تسريب ميموري (Memory Leak).
٢. إيه اللي حصل لما ظهر الـ .NET؟

لما في gather أو لما Microsoft عملت الـ .NET Framework، قالوا: "ليه
بتعب المبرمج ونخليه يشيل هم الميموري وتوافق الأجهزة؟"
فهنا عملية الترجمة بقت على مرحلتين:

المرحلة الأولى (وأنت بتبني المشروع): الكود بتاعك (C# مثلاً) بيتترجم للغة
وسطية اسمها CLR (حاجة كده زي لغة المهايدين، مش بتاعة بروسيسور
معيّن) وبتحطها جوه ملف exe أو dll.

المرحلة الثانية (وأنت بتشغل البرنامج): بيحي دور البيئة التشغيلية اللي
اسمها CLR (ده المايسټرو المحرك)، وفيه جواها مترجم فوري اسمه JIT (Just-In-Time)
(بياخد الكود الوسطي ده وبترجمه للغة البروسيسور لحظة
التشغيل بالضبط).

الفوائد اللي كسبتها:

Garbage Collector (اللعام): محرك الـ .NET بينصف الميموري وراها أول
بأول، مش محتاج تفكر تقضي الـ RAM بنفسك.

الأمان: الكود بيمشي تحت حراسة الـ CLR، فصعب يعمل مشاكل برة مساحته
المسموح بيها.

اللغات بتفهم بعض: ممكن تكتب جزء بلغة C# وجزء بـ VB.NET واللاتين
يكلموا بعض عادي جداً لأنهم بيتترجموا لنفس اللغة الوسطية!

SoftwareEngineering #DotNet #CSharp #Programming #CleanCode
ode #WebDevelopment #SoftwareArchitecture #DeveloperComm
unity

1. Evolution of .NET Versions

.NET Framework (Old): Made for Windows only. It cannot run on Linux or Mac.

.NET Core (2016): Built from scratch to be Cross-Platform (works on Windows, Linux, and Mac), fast, and open-source.

Modern .NET (.NET 5 / 6 / 8+): Microsoft combined everything into one unified system with higher performance.

2. What is a Namespace?

A Namespace is a logical container used to organize your code and keep things clean.

Main Goal: It prevents name conflicts (for example, having two different classes both named User).

How to use it: You use the using keyword at the top of your code file to easily access classes inside a namespace.

3. Difference Between Project and Solution

Project (.csproj): Represents a single application or library (like a Web API or Console App) that compiles into one executable or assembly file (.dll or .exe).

Solution (.sln): Acts as the master container that holds multiple related projects together so you can build, manage, and run them as a complete system.